

Beambot - Beamline Robotics Documentation

Aditya Bondada

January 10, 2026

1 Introduction

1.1 The Challenge: Sample Handling at Synchrotron Beamlines

1.2 Vision: Self-Driving Beamlines

1.3 The Gap: Integration Challenges

1.4 This Work: Beambot

1.5 Document Structure

2 Background

2.1 Synchrotron Beamlines

2.2 Robotic Manipulation

2.3 Existing Solutions

2.4 Technology Choices

3 System Architecture

3.1 High-Level Architecture

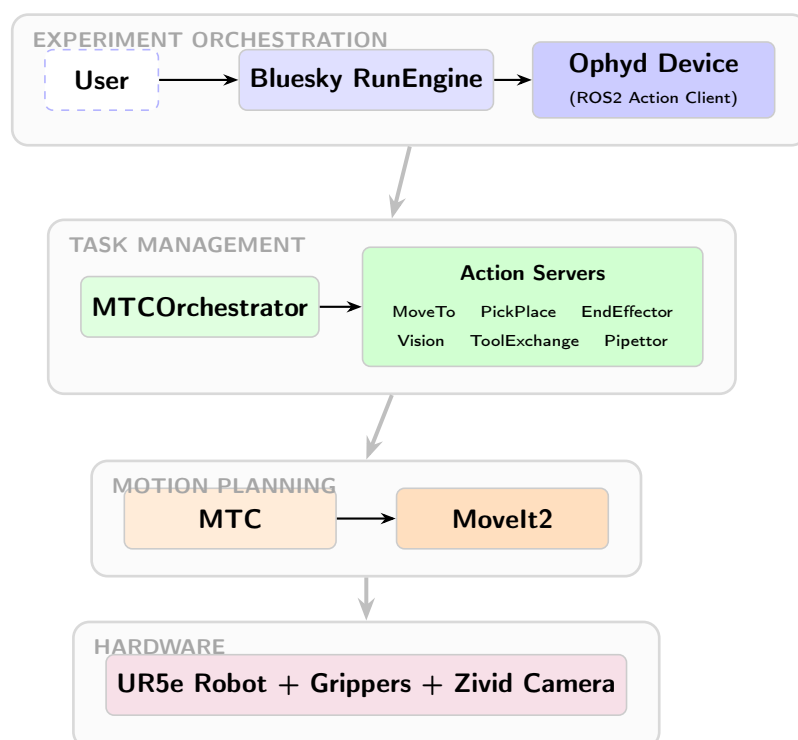


Figure 1: Beambot software architecture: horizontal data flow within layers, vertical flow between architectural layers.

The system architecture is organized into four primary layers, each responsible for distinct functionalities within the overall system (see Figure 1). The user / Beamline scientist interacts with the Bluesky RunEngine to define and execute experiments. This is the same interface used to control other beamline hardware, ensuring seamless integration into existing workflows. To facilitate this the ROS2 action client is wrapped as an Ophyd device - so it behaves like any other beamline hardware component. The RunEngine sends high-level commands to the MTCOrchestrator, which manages task execution by coordinating various action servers. These action servers encapsulate specific functionalities such as moving the robot, picking and placing samples, vision processing, tool exchanges, and pipetting operations. The MTC (Motion Task Compiler) layer translates high-level task commands into motion plans, leveraging the MoveIt2 framework for path planning and collision avoidance. Finally, the hardware layer consists of the UR5e robotic arm, equipped with grippers and a Zivid 3D camera for visual feedback.

3.2 Experiment Orchestration Layer

The user / Beamline scientist interacts with the Bluesky RunEngine to define and execute experiments. The Ophyd device/ROS2 action client sends a JSON file that defines the experimentation task sequence - e.g., load sample from position A, move to sample place position, change tool, etc.

3.3 Task Management Layer

3.4 Motion Planning Layer

3.5 Hardware Layer